

SAC-FTOPSIS: Um Modelo de Decisão para o Descarregamento de Tarefas em Redes Veiculares

Antônio Sérgio de Sousa Vieira^{1,2*}, Alisson Barbosa de Souza^{1,3} and Joaquim Celestino Júnior¹

^{1*}Grupo de Redes e Sistemas Inteligentes (INETSYS), Universidade Estadual do Ceará, Fortaleza, Brasil.

²Instituto Federal de Educação, Ciência e Tecnologia, Itapipoca, Brasil.

³Universidade Federal do Ceará, Quixadá, Brasil.

*Corresponding author(s). E-mail(s): sergio.vieira@ifce.edu.br;
Contributing authors: alisson@ufc.br; joaquim.celestino@uece.br;

Resumo

O descarregamento de tarefas em redes de Computação de Borda Veicular (VEC) é um mecanismo crítico para satisfazer as rigorosas restrições de latência e energia de aplicações autônomas avançadas. A alta mobilidade dos veículos cria um ambiente no qual o número de servidores de borda e vizinhos disponíveis muda continuamente, tornando o espaço de observação dimensionalmente variável. Essa variabilidade impede que agentes padrão de Aprendizado por Reforço Profundo (DRL) generalizem entre diferentes densidades veiculares sem retreinamento custoso. Para enfrentar esse desafio, este artigo propõe o **SAC-TOPSIS**, uma arquitetura híbrida de tomada de decisão que integra a Técnica de Ordenação por Preferência à Similaridade com a Solução Ideal (Técnica de Ordenação por Preferência à Similaridade com a Solução Ideal (TOPSIS)) ao algoritmo Soft Actor-Critic (SAC). O módulo TOPSIS avalia cada nó candidato ao descarregamento, caracterizado por Tempo de Conclusão de Tarefa (JCT), consumo de energia e distância física, usando distâncias L_1 -Manhattan com pesos **[0,4; 0,3; 0,3]**, e comprime a observação de tamanho variável em um vetor de estado fixo de 6 dimensões, invariante à escala. Um agente SAC recebe essa representação compacta e aprende uma política binária: executar localmente ou descarregar para o candidato melhor ranqueado. A regularização por entropia impede que a política colapse em modos puramente locais ou puramente de descarregamento. Simulações extensivas em 601 cenários heterogêneos Vehicle-to-Everything (V2X) e 5G demonstram que a integração do TOPSIS melhora consistentemente todas as famílias de DRL testadas: SAC melhora de **4,47%** para **44,36%** (**+39,9 pp**), Twin Delayed DDPG (TD3) de **41,62%** para **43,01%** (**+1,4 pp**) e Proximal Policy Optimization (PPO) de **41,61%** para **41,95%** (**+0,3 pp**) na taxa de cumprimento de prazo. Entre todas as políticas avaliadas, o SAC-TOPSIS alcança a maior taxa de cumprimento (**44,36%**) e o maior índice de Qualidade Ponderada pelo Sucesso (SWQ) (**0,1450**). Fundamentalmente, o modelo é treinado em um único regime de densidade e generaliza zero-shot para densidades veiculares inéditas de 10 a 30 tasks/s.

Keywords: Descarregamento de tarefas, Computação de Borda Veicular, Soft Actor-Critic, TOPSIS, Compressão de estado multicritério, Generalização zero-shot

1 Introdução

As redes de comunicação veicular passaram por algumas transições tecnológicas importantes. Inicialmente, o foco estava nas Redes Veiculares Ad hoc (VANETs), que priorizavam a conectividade direta entre veículos, mas enfrentavam desafios relacionados à latência e à conectividade devido à alta mobilidade dos nós [1]. Com o amadurecimento da Internet das Coisas (IoT) e a expansão de sensores embarcados na infraestrutura urbana, desenvolveu-se um paradigma de rede de grande escala que integra veículos, infraestrutura, pedestres e a nuvem, conhecido como ecossistema da Internet dos Veículos (IoV) [1]. Tal ecossistema foi idealizado para viabilizar os Sistemas de Transporte Inteligentes (ITS) verdadeiramente autônomos e conectados, possibilitando às aplicações veiculares atender simultaneamente a requisitos de eficiência, confiabilidade e segurança [2].

No entanto, a IoV depende da capacidade de atendimento aos requisitos de baixa latência e alta vazão das tecnologias de rádio. Dessa forma, a comunicação veicular passou por algumas evoluções, partindo inicialmente da alocação de 75 MHz na banda de 5.9 GHz para o padrão Dedicated Short-Range Communications (DSRC), que definiu os parâmetros para comunicação Vehicle-to-Vehicle (V2V) e Vehicle-to-Infrastructure (V2I) nas primeiras VANETs [3]. A partir desse legado, desenvolveram-se dois eixos tecnológicos de comunicação. O primeiro, o IEEE 802.11bd (Next-Generation V2X), estendeu o IEEE 802.11p incorporando robustez contra desvanecimento rápido e suporte a velocidades de até 250 km h^{-1} [4]. Já o segundo direciona-se à superação das limitações do DSRC em escalabilidade e custo de implantação, o que impulsionou o desenvolvimento do Cellular V2X (C-V2X) pelo 3GPP [5]. Sob a perspectiva regulatória, a disputa para adesão dessas tecnologias foi resolvida pela Federal Communications Commission (FCC), que estabeleceu o C-V2X como padrão obrigatório para a comunicação de ITS, com a descontinuação oficial do DSRC marcada para dezembro de 2026 [6].

A evolução das tecnologias de comunicação conseguiu atender aos requisitos de latência de rede e vazão dos ITS. No entanto, as limitações

em relação à capacidade computacional dos dispositivos embarcados ainda precisavam ser superadas. Nesse contexto, a Computação de Borda de Múltiplos Acessos (MEC) e sua especialização veicular, a Computação de Borda Veicular (VEC), viabilizam o funcionamento de aplicações sensíveis à latência ao posicionarem recursos de processamento e armazenamento próximos à rede de acesso por rádio [7, 8]. Dessa forma, os usuários finais podem transferir tarefas computacionalmente intensivas para os servidores de borda utilizando a infraestrutura 5G C-V2X. Essa delegação reduz drasticamente o tempo de resposta e o consumo de energia a bordo [9, 10, 11], garantindo, assim, a Qualidade de Serviço (QoS) de aplicações sensíveis à latência.

Em ambientes veiculares, as capacidades computacionais das Unidades de Bordo Interconectadas (OBUIs) são inerentemente heterogêneas [12]. Essa diversidade de hardware, se bem aproveitada, possibilita a formação de ambientes de execução cooperativos, nos quais veículos equipados com unidades de processamento de alto desempenho disponibilizam sua capacidade ociosa temporária para auxiliar nós vizinhos com restrições computacionais e energéticas. O compartilhamento direto desses recursos melhora consideravelmente a QoS global dos ITS [13]. Dessa forma, a integração da alta capacidade do servidor de borda com a disponibilização de processamento ocioso das OBUIs fecha o ciclo de melhorias necessário para viabilizar a execução de aplicações veiculares computacionalmente intensivas e sensíveis à latência e ao consumo energético.

A operacionalização desse potencial colaborativo tem como mecanismo central o descarregamento de tarefas. Essa estratégia consiste em transferir a responsabilidade de computar tarefas complexas de dispositivos com recursos limitados para nós de processamento remoto mais poderosos, como o servidor de borda ou veículos vizinhos com maior capacidade de processamento [14]. O objetivo é contornar as restrições locais de bateria, armazenamento e processamento com o intuito de reduzir o consumo de energia, minimizar a latência e melhorar a QoS. O grande desafio, contudo, reside em decidir dinamicamente quando e para onde descarregar essas tarefas. Explorar plenamente esse potencial exige o desenvolvimento de políticas de alocação capazes de se adaptar à topologia altamente dinâmica da rede e de avaliar

múltiplos critérios de decisão de forma simultânea e em tempo real.

O descarregamento de tarefas em redes veiculares configura-se fundamentalmente como um problema de otimização multiobjetivo [15, 16]. Nesse contexto, diversas políticas têm sido propostas para solucionar esse problema. As heurísticas baseadas em regras lógicas e os métodos de Tomada de Decisão Multicritério (MCDM), a exemplo das estratégias gulosas, são amplamente utilizados devido à sua simplicidade computacional e facilidade de implementação [17]. No entanto, essas abordagens avaliam apenas a situação instantânea da rede, mostrando-se incapazes de antecipar as dinâmicas de tráfego e os estados futuros do ambiente. Por outro lado, as meta-heurísticas, tais como algoritmos genéticos, otimização por enxame de partículas e colônia de formigas, oferecem garantias mais sólidas de busca global e têm sido aplicadas com sucesso na tomada de decisão de descarregamento em cenários estáticos [18, 19, 20]. Contudo, esses métodos podem se mostrar inadequados para os ambientes veiculares, visto que a busca baseada em população exige dezenas a centenas de avaliações de aptidão iterativas a cada instante de decisão, podendo causar atrasos inaceitáveis [21, 22]. Essa característica de convergência relativamente lenta pode ser incompatível com os rigorosos prazos de execução na escala de milissegundos característicos das aplicações veiculares [8, 2]. Além disso, esses algoritmos precisam ser reiniciados a cada alteração topológica do ambiente, e a sua complexidade de inicialização e convergência cresce significativamente com o aumento do tamanho da rede, podendo degradar a qualidade da solução em cenários de larga escala [15, 11].

Cenários de avaliação simplificados, com número fixo de veículos, distribuições de tarefas uniformes e modelos de canal idealizados, não capturam a complexidade e o alto dinamismo das redes veiculares reais [8]. Políticas de descarregamento treinadas e avaliadas nesses ambientes podem sofrer degradação de desempenho quando a densidade veicular ou a distribuição da carga de trabalho muda repentinamente [8], revelando uma dicotomia crítica entre o sucesso obtido em simulações e a real aplicabilidade em cenários operacionais. Nesse contexto, é fundamental que o desenvolvimento de políticas de descarregamento

robustas busque ativamente mitigar essa disparidade. Isso exige a concepção de representações de estado adaptativas que garantam a capacidade de generalização do modelo perante as constantes alterações no número de nós candidatos ao descarregamento eliminando a dependência de ambientes de tráfego estacionários [23, 2].

O DRL apresenta-se como uma solução natural para o descarregamento de tarefas porque aprende uma política de controle que mapeia observações em ações considerando as consequências futuras de longo prazo, e não apenas o estado instantâneo da rede [9, 24]. Diferentemente das heurísticas e das meta-heurísticas, um agente DRL possui a capacidade de explorar correlações temporais nos padrões de tráfego, na dinâmica das filas, na mobilidade veicular e na qualidade do canal de comunicação a partir da interação contínua com o ambiente [25, 2].

No entanto, as arquiteturas DRL padrão compartilham uma fraqueza estrutural crítica relacionada à sua aplicabilidade prática, pois frequentemente exigem retreinamento sempre que o cenário operacional sofre mudanças na quantidade de nós computacionais candidatos [8]. Como a camada de entrada das redes neurais profundas depende de um vetor de tamanho fixo, as constantes alterações no número de nós candidatos ao descarregamento, como o servidor de borda e veículos vizinhos, acabam invalidando o modelo previamente treinado [2].

Neste contexto, esse trabalho propõe uma política híbrida de descarregamento de tarefas denominada SAC-TOPSIS capaz de lidar com as restrições de funcionamento das DRL em um ambiente veicular dinâmico. Em resumo, as principais contribuições são:

1. Desenvolvimento de uma solução inovadora para o descarregamento de tarefas que combina a técnica TOPSIS com o algoritmo SAC. O TOPSIS fornece a melhor alternativa baseada nas condições instantâneas da rede, enquanto o SAC aprende a política ótima de longo prazo por meio da experiência acumulada.
2. Representação de um conjunto de tamanho variável de nós candidatos ao descarregamento em um vetor de estado compacto de seis dimensões na qual a política de controle treinada em um cenário representativo pode ser testada em ambientes inéditos sem a necessidade de retreinamento.

3. Redução do espaço de ações a uma decisão binária (processamento local vs. melhor candidato de descarregamento), possibilitando a inferência em $O(1)$ passagens diretas na rede neural por tarefa, tornando o modelo adequado para a implantação embarcada em tempo real.
4. Desenvolvimento de uma função de recompensa sensível ao prazo que fornece um sinal de gradiente contínuo em ambos os regimes de sucesso e falha, evitando a esparsidade de recompensa e ensinando o agente a minimizar ativamente o atraso em vez de meramente evitá-lo.

O restante deste artigo está organizado da seguinte forma. A Seção 2 revisa os trabalhos relacionados presentes na literatura. A Seção 3 formaliza o modelo do sistema e define o objetivo de otimização. A Seção 4 detalha a arquitetura da política híbrida SAC-TOPSIS proposta. A Seção 5 apresenta a avaliação experimental e discute os resultados obtidos. A Seção 6 conclui o artigo, apresentando as considerações finais e direções para trabalhos futuros. Além disso, as definições de todas as siglas utilizadas ao longo do texto encontram-se sumarizadas na Tabela 1.

2 Trabalhos Relacionados

A otimização do descarregamento de tarefas em redes veiculares representa um desafio crítico para viabilizar sistemas de transporte inteligentes, exigindo decisões rápidas que equilibrem latência e consumo de energia. O avanço das arquiteturas de Computação de Borda Veicular (VEC) transferiu a capacidade de processamento para mais perto dos usuários, mas a alta mobilidade e a variação da carga de rede tornam as heurísticas estáticas ineficientes. Por conta disso, a literatura recente tem transitado de métodos matemáticos tradicionais para abordagens baseadas em Aprendizado por Reforço Profundo (DRL), buscando políticas de alocação que se adaptem dinamicamente ao ambiente.

O trabalho apresentado em [26] desenvolveu o framework FOFF fundamentado em Tomada de Decisão Multicritério (MCDM). A solução aplicou a técnica Fuzzy-TOPSIS para ranquear o servidor de borda (VEC) e veículos vizinhos com base no tempo de resposta e consumo de energia. As simulações demonstraram que a abordagem obteve economias significativas de energia sem a necessidade de uma fase de treinamento prévio.

Contudo, modelos baseados exclusivamente em regras lógicas ou avaliações heurísticas falham diante de cenários veiculares não estacionários, visto que não conseguem antecipar flutuações nas dinâmicas de fila ou aprender com o histórico de desempenho.

O estudo conduzido por [27] explorou o aprendizado de máquina para lidar com a mobilidade dos usuários em redes veiculares. A pesquisa propôs um framework baseado em Redes Q Profundas (DQN) para decidir a alocação em unidades de beira de estrada e gerenciar o descarregamento de tarefas. Os resultados indicaram uma redução efetiva nos custos de comunicação e cálculo. A principal limitação dessa abordagem reside em sua natureza centralizada, exigindo o controle simultâneo de todos os veículos da rede e provocando um aumento exponencial na complexidade computacional à medida que a densidade veicular cresce.

O trabalho de [28] desenvolveu um algoritmo de descarregamento multitarefa utilizando a arquitetura Dueling Double Deep Q-Network (D3QN). O método focou na minimização do tempo de resposta e do consumo energético em ambientes de borda altamente requisitados. As avaliações mostraram que o modelo reduziu a latência do sistema de forma mais eficaz que abordagens aleatórias tradicionais. No entanto, por ser baseado em espaços de ação estritamente discretos, o algoritmo apresenta dificuldades de escalabilidade quando aplicado a problemas de alocação contínua de recursos, limitando a precisão da distribuição de poder computacional.

A pesquisa apresentada por [29] propôs uma solução de descarregamento baseada em um jogo de Stackelberg de quatro estágios integrado a um modelo de aprendizado por reforço. A estratégia avalia o estado do veículo em tempo real para otimizar o tempo e a despesa do descarregamento de forma coordenada. Os experimentos confirmaram que a solução alcançou um equilíbrio na utilidade do sistema e melhorou a probabilidade de sucesso das tarefas. Apesar dos ganhos, a dependência de processos de negociação complexos entre múltiplos agentes introduz uma sobrecarga de sinalização que degrada o desempenho em cenários de tráfego denso e comunicação intermitente.

O estudo de [24] buscou resolver o problema de alocação de recursos contínuos aplicando o algoritmo Ator-Crítico Determinístico Profundo

Tabela 1 Lista de Acrônimos – Resumo das principais siglas utilizadas ao longo deste artigo.

Sigla	Definição	Sigla	Definição
3GPP	3rd Generation Partnership Project	NIS	Solução Ideal Negativa
5G	5th Generation	NR-V2X	New Radio Vehicle-to-Everything
C-V2X	Cellular Vehicle-to-Everything	OBUI	Unidades de Bordo Interconectadas
D3QN	Dueling Double Deep Q-Network	PIS	Solução Ideal Positiva
DRL	Aprendizado por Reforço Profundo	QoS	Qualidade de Serviço
DSRC	Dedicated Short-Range Comm.	RSU	Roadside Unit
IEEE	Inst. of Elect. and Electr. Eng.	SAC	Soft Actor-Critic
IoT	Internet of Things	TOPSIS	Tech. Order Pref. Sim. Ideal Sol.
IoV	Internet de Veículos	V2I	Vehicle-to-Infrastructure
ITS	Sistemas Transporte Inteligentes	V2N	Vehicle-to-Network
JCT	Tempo de Conclusão de Tarefa	V2P	Vehicle-to-Pedestrian
MCDM	Tomada de Decisão Multicritério	V2V	Vehicle-to-Vehicle
MEC	Computação de Borda de Múltiplos Acessos	VEC	Computação de Borda Veicular

(DDPG). A abordagem utilizou a estrutura ator-crítico para reduzir o custo geral do sistema em um ambiente colaborativo de nuvem e borda veicular. O algoritmo convergiu rapidamente e reduziu os custos operacionais em comparação com métodos baseados apenas em valor. A falha dessa modelagem é a ausência de consideração sobre a dependência estrutural entre as tarefas e a utilização de preenchimento com zeros na observação do estado para lidar com o número dinâmico de veículos, obrigando a rede neural a processar artefatos espúrios e ruídos.

O trabalho desenvolvido por [30] implementou um esquema adaptativo usando aprendizado por reforço multiagente em computação em névoa veicular. A solução distribuiu a tomada de decisão diretamente entre os veículos para reduzir a latência e o consumo de energia de forma descentralizada. As simulações demonstraram superioridade na capacidade de adaptação local em comparação com arquiteturas guiadas por agentes únicos. A limitação crítica dessa arquitetura é o desafio da sincronização e a dificuldade de garantir uma convergência estável quando múltiplos veículos alteram o ambiente simultaneamente, gerando instabilidade na política global de roteamento.

O estudo conduzido por [31] integrou a tecnologia de gêmeos digitais a redes de veículos para habilitar o descarregamento proativo assistido por inteligência artificial. O trabalho empregou o algoritmo Asynchronous Advantage Actor-Critic (A3C) para otimizar as decisões baseando-se em uma réplica virtual contínua do ambiente físico veicular. A pesquisa relatou uma convergência rápida da rede e a diminuição considerável do

custo do sistema. A restrição fundamental desse modelo é a altíssima sobrecarga computacional exigida para manter o gêmeo digital sincronizado em tempo real, fator que frequentemente inviabiliza sua execução fluida em dispositivos móveis.

A pesquisa de [32] direcionou o aprendizado por reforço para tarefas de visão computacional e propôs um algoritmo focado na detecção de objetos em veículos autônomos. O modelo combinou DRL com linearização por partes para maximizar especificamente uma função de utilidade baseada em tempo de execução. Os resultados apontaram melhorias expressivas no tempo de resposta para o processamento de imagens críticas. Contudo, a modelagem foi construída de forma monolítica para um tipo específico de aplicação, falhando em considerar as prioridades de diferentes tarefas concorrentes e negligenciando os problemas de congestionamento causados pela densidade veicular.

O estudo apresentado por [33] focou no descarregamento de tarefas complexas modeladas como grafos direcionados acíclicos utilizando o algoritmo Ator-Crítico Suave (SAC). O framework buscou equilibrar ativamente o atraso e o gasto de energia para aplicações com alta dependência estrutural entre seus subcomponentes. A avaliação atestou que a maximização da entropia estocástica evitou a convergência para ótimos locais e melhorou a utilidade de execução. A restrição principal desse modelo para o contexto automotivo é a premissa de um ambiente perfeitamente estático, ignorando completamente as taxas de desconexão e a variabilidade do canal de comunicação.

Uma abordagem híbrida foi desenvolvida por [21], que propôs o esquema TOLB para descarregamento e balanceamento de carga integrando o Twin Delayed Deep Deterministic Policy Gradient (TD3) com a Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS). A metodologia utilizou o algoritmo de aprendizado para decisões de proporção de particionamento e o cálculo multicritério para selecionar o servidor alvo. Os experimentos comprovaram a redução do atraso de processamento e a diminuição do desequilíbrio de carga na infraestrutura de borda. A limitação dessa abordagem reside no uso da filtragem multicritério apenas como um elemento de pós-processamento tático, mantendo o espaço de observação bruto e engessado na entrada do agente e prejudicando a generalização espacial em redes densas.

A arquitetura SAC-TOPSIS proposta nesta etapa intermediária difere substancialmente dessas abordagens, uma vez que consolida o método TOPSIS internamente na borda de observação atuando estritamente como um módulo de engenharia de estado. Esse módulo avalia o servidor de borda (VEC) e veículos vizinhos com base no Tempo de Conclusão de Tarefa (JCT), consumo energético e distância ao nó de computação para identificar o melhor candidato ao descarregamento. A saída computada condensa essas características em uma codificação de estado contínua estritamente compacta como um vetor 6D perfeitamente invariante à escala, eliminando a dependência do preenchimento com zeros. A avaliação experimental comparou cinco famílias de DRL (SAC, Otimização de Política Proximal (PPO), TD3, DQN e Advantage Actor-Critic (A2C)) e sete heurísticas de referência (VEC, Random, Round-Robin, Gini, Jain, Greedy-Fair e Greedy-Unfair), atestando que o emparelhamento desse espaço estruturado produz ganhos consistentes. Isso valida a tese central de que a engenharia composicional do estado elaborada via TOPSIS atua como o principal motor impulsionador da capacidade de generalização apresentada pelo agente inteligente em ambientes de tráfego variável. Em resumo, os principais aspectos dos trabalhos relacionados são apresentados na Tabela 2.

3 Modelo de Sistema e Formulação do Problema

Nesta seção, são apresentados os principais modelos matemáticos que fundamentam a solução proposta, englobando as dinâmicas de descarregamento de tarefas e latência, a dinâmica térmica (*thermal throttling* e resfriamento), o consumo energético, a comunicação celular e veicular, e a definição da função objetivo. Para facilitar a leitura e a compreensão formal, a Tabela 3 sumariza as principais notações empregadas.

3.1 Modelo de Tarefa, Latência e Dinâmica Térmica

Cada tarefa é caracterizada pelo par $\mathcal{T}_i = (c_i, d_i)$, onde c_i representa a complexidade da tarefa (em ciclos de CPU) e d_i denota o prazo aceitável para sua conclusão [9, 2, 34, 22]. A decisão de descarregamento de tarefas é binária (processamento local ou remoto) com o objetivo de reduzir a complexidade computacional da política de controle, o que favorece a sua aplicabilidade em sistemas de tempo real [9, 29].

A latência de execução é modelada pela proporção entre a carga de trabalho total (c_i) e a capacidade de processamento do dispositivo (f^m). Consequentemente, o tempo de resposta para o processamento da tarefa i no modo selecionado $m \in \{local, remoto\}$ é expresso pela Equação 1 e o tempo total para a conclusão da tarefa (JCT) é dado pela Equação 2:

$$L_i^m = \frac{c_i}{f^m} \quad (1)$$

$$JCT_i = t_i^{\text{upload}} + t_i^{\text{fila}} + L_i^m + t_i^{\text{download}}, \quad (2)$$

em que t_i^{upload} é o tempo de transmissão da tarefa i , t_i^{fila} é o tempo de espera residual na fila de processamento (estimado como o tempo de execução restante das tarefas já agendadas), L_i^m é o tempo de execução da tarefa no modo selecionado m e t_i^{download} é o tempo necessário para o download dos resultados. Para o modo de processamento local, os tempos de transmissão de *upload* e *download* são nulos.

Para que o cálculo do JCT seja mais realista, é necessário considerar o impacto do mecanismo de segurança térmica do dispositivo. A execução

Tabela 2 Comparação das Soluções de Descarregamento – Uma avaliação comparativa das estratégias de descarregamento propostas em estudos anteriores.

Ref.	Estratégia	Tecnologia de Comunicação	Conquista	Limitação
[26]	Fuzzy-TOPSIS	VEC	Economias significativas de energia sem fase de treinamento prévio	Falha em adaptar-se a cenários veiculares não estacionários
[27]	DQN	Redes Veiculares	Redução efetiva nos custos de comunicação e cálculo	Aumento exponencial na complexidade devido à natureza centralizada
[28]	D3QN	IoV e MEC	Redução da latência do sistema de forma mais eficaz	Dificuldade de escalabilidade em problemas de alocação contínua
[29]	DRL + Jogo de Stackelberg	Redes Veiculares	Equilíbrio na utilização do sistema e sucesso das tarefas	Alta sobrecarga de sinalização em cenários de tráfego denso
[24]	DDPG	Nuvem e Borda Veicular	Convergência rápida e redução de custos operacionais	Utilização de preenchimento com zeros e processamento de ruídos
[30]	MARL	Névoa Veicular	Superioridade na capacidade de adaptação local descentralizada	Desafio de sincronização e convergência instável na rede global
[31]	A3C + Gêmeos Digitais	IoV	Convergência rápida da rede e diminuição do custo do sistema	Altíssima sobrecarga computacional exigida para sincronização
[32]	DRL + Linearização por partes	Veículos Autônomos	Melhorias expressivas no tempo de resposta para processamento de imagens	Negligência problemas de congestionamento e prioridades de tarefas
[33]	SAC	Nuvem Veicular	Evita a convergência para ótimos locais e melhora a utilidade	Premissa de ambiente estático e ignorância da variabilidade do canal
[21]	TD3 + TOPSIS	VEC	Redução do atraso de processamento e do desequilíbrio de carga	Filtragem multicritério utilizada apenas como pós-processamento tático

Tabela 3 Resumo das Notações Utilizadas

Símbolo	Descrição
<i>Modelo de Tarefa</i>	
$\mathcal{T}_i$	Tarefa i definida por (c_i, d_i)
c_i	Complexidade computacional da tarefa i (em ciclos)
d_i	Prazo máximo (<i>deadline</i>) da tarefa i (em s)
m	Modo de execução (l : local, r : remoto)
f^m	Frequência de operação da CPU no modo m (em Hz)
<i>Dinâmica Térmica e Energia</i>	
P_{cpu}^m	Potência elétrica consumida pela CPU (em W)
k_1, k_2, k_3	Coefficientes de potência (dinâmica, fuga e estática)
$T_p(t)$	Temperatura do <i>chip</i> no instante t (em °C)
T_{amb}	Temperatura ambiente externa (em °C)
$T_{p,\text{max}}$	Limite de temperatura crítico (<i>throttling</i>) (em °C)
R'_p, C_p	Resistência (°C W ⁻¹) e capacitância (J °C ⁻¹) térmicas
<i>Métricas e Otimização</i>	
L_i^m	Tempo de processamento da tarefa no modo m (em s)
JCT_i	Tempo total de conclusão da tarefa (em s)
E_i^m	Energia total consumida para processar a tarefa i (em J)
Φ_i	Contribuição de QoS da tarefa i
α, β, γ	Pesos de latência, energia e penalidade de prazo

contínua de tarefas eleva a temperatura do processador, o que pode forçar o sistema a reduzir a frequência operacional f^m para evitar o superaquecimento físico do dispositivo.

Nesse contexto, a potência elétrica dissipada pela CPU ($P_{\text{cpu}}(f^m)$) é calculada pelo modelo polinomial de frequência proposto por Silva et al. [35], dada pela Equação 3:

$$P_{\text{cpu}}(f^m) = k_1(f^m)^3 + k_2f^m + k_3 \quad (3)$$

em que k_1 e k_2 representam as constantes de potência dinâmica e de fuga de corrente (*leakage*), respectivamente. A constante k_3 indica o consumo estático mínimo da arquitetura.

A temperatura instantânea do processador $T_p(t)$, após um intervalo de tempo sob uma

dissipação de potência constante $P_{\text{cpu}}(f^m)$, segue o modelo de circuito térmico RC de Rao e Vrudhula [36], expresso pela Equação 4:

$$T_p(t) = P_{\text{cpu}} \cdot R'_p + T_{\text{amb}} + (T_{p0} - (P_{\text{cpu}} \cdot R'_p + T_{\text{amb}}))e^{-\frac{t}{R'_p C_p}} \quad (4)$$

em que T_{p0} é a temperatura no instante inicial, T_{amb} é a temperatura ambiente, e R'_p e C_p são parâmetros que representam a resistência e a capacitância térmicas do CPU, respectivamente.

Essa modelagem permite representar de forma mais realista os ciclos de aquecimento e resfriamento do processador. No estado de aquecimento, a potência exigida ($P_{\text{cpu}} = P'_{\text{max}}$) é alta, gerando um aumento exponencial na temperatura. Para proteger o processador, o modelo projeta matematicamente o instante exato t_m em que a temperatura atingirá o limite crítico ($T_{p,\text{max}}$). Derivando a Equação 4, obtém-se a restrição de segurança (Equação 5):

$$t_m = R'_p C_p \ln \left(\frac{P'_{\text{max}} R'_p + T_{\text{amb}} - T_{p0}}{P'_{\text{max}} R'_p + T_{\text{amb}} - T_{p,\text{max}}} \right) \quad (5)$$

Se o tempo de processamento exceder t_m , a temperatura fatalmente alcançará a fronteira máxima $T_{p,\text{max}}$. Nesse limiar, o mecanismo de contenção térmica (*thermal throttling*) é acionado, forçando a redução da frequência f^m de pico para uma frequência de estabilização térmica segura, freando a curva de dissipação de calor, ao custo de estender significativamente a latência L_i^m .

Quando a fila de processamento esvazia, a CPU cessa a atividade computacional e f^m zera de forma virtual. Nesse estado, a potência dissipada cai de P'_{max} para $P_{\text{cpu}} = k_3$, que representa unicamente o consumo estático da arquitetura. Inserindo essa potência mínima subtrativa na Equação 4, a temperatura assintótica cai drasticamente, tornando o diferencial de resfriamento ($T_{p0} - (k_3 \cdot R'_p + T_{\text{amb}})$) fortemente positivo. Portanto, o decaimento exponencial age como amortecedor, reduzindo continuamente a temperatura $T_p(t)$ em direção ao novo limiar basal seguro de repouso durante todo o tempo do *intervalo idle*.

3.2 Modelo de Energia

Dessa forma, a energia total consumida E_i^m durante todo o ciclo de vida da tarefa i no modo $m \in \{l, r\}$ engloba de forma coerente a potência instantânea do processamento e o custo das interfaces de rede, conforme a Equação 6:

$$E_i^m = P_{\text{cpu}}^m(f^m) \cdot L_i^m + P_{\text{tx}}^m \cdot t_i^{\text{trans},m} \quad (6)$$

onde P_{tx}^m denota a potência de transmissão constante e $t_i^{\text{trans},m}$ representa o tempo total de transmissão da tarefa (englobando *upload* e *download*). Para o modo de processamento local ($m = l$), a interface de rede não é acionada, tornando o termo de transmissão estritamente nulo ($P_{\text{tx}}^l = 0$).

3.3 Modelo de Comunicação V2I e V2V

A modelagem da comunicação V2I e V2X segue as definições de [37, 38]. O enlace V2I opera na frequência de 3.5 GHz com largura de banda de 50 MHz, potência de transmissão de 23 dBm e antenas assimétricas (1.5 m nos veículos e 25 m na estação rádio base). O valor de desvanecimento adotado é de 4 dBm com um atraso de processamento (*jitter*) de 0.1 ms. Para a interface V2X (*Sidelink*), a frequência central é de 5.9 GHz, com largura de banda de 10 MHz, desvanecimento de 3 dBm e atraso de processamento de 0.25 ms. Ambas as interfaces adotam o modelo urbano sem linha de visada (Urban NLOS) do 3GPP [37], no qual a perda de percurso (**PL**!, do inglês *Path Loss*) expressa em dB é determinada pela Equação 7:

$$PL(d, f) = 35,3 \log_{10}(d) + 21,3 \log_{10}(f) + 22,4 \quad (7)$$

onde d representa a distância tridimensional entre o transmissor e o receptor em metros, e f é a frequência de operação em GHz. Esses parâmetros garantem uma representação realista da atenuação de sinal durante a troca de dados entre os nós candidatos ao descarregamento, como o servidor de borda (VEC) e veículos vizinhos.

SAC-TOPSIS Architecture: TOPSIS State Compression + Maximum-Entropy Policy

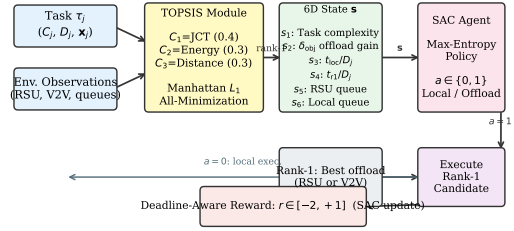


Figura 1 Arquitetura SAC-TOPSIS. O TOPSIS comprime o conjunto de candidatos de tamanho variável em um vetor de estado fixo 6D invariante à escala. O SAC aprende uma política binária (local vs. melhor descarregamento) sobre essa representação com regularização de máxima entropia.

3.4 Objetivo de Otimização

A formulação da contribuição de QoS para a tarefa i é expressa pela Equação 8:

$$\Phi_i = \alpha \cdot \text{JCT}_i + \beta \cdot E_i + \gamma \cdot \mathbf{1}[\text{JCT}_i > d_i], \quad (8)$$

que incorpora parâmetros configuráveis de peso (por padrão, $\alpha = \beta = 0,35$) e uma constante de penalidade rigorosa por violação de prazo ($\gamma = 0,015$). O objetivo global do sistema é minimizar o custo agregado $\sum_i \Phi_i$ enquanto maximiza ativamente a taxa de cumprimento de prazo, definida como $\hat{\tau} = |\{i : \text{JCT}_i \leq d_i\}|/N$.

4 A Arquitetura SAC-TOPSIS

A Figura 1 ilustra a arquitetura completa. A cada instante de decisão, o módulo TOPSIS processa as informações multicritério sobre todos os candidatos viáveis e produz o vetor de estado 6D que é passado ao agente SAC.

4.1 Compressão de Estado Baseada em TOPSIS

Matriz de decisão.

Para a tarefa τ_j , construímos uma matriz de decisão $\mathbf{X} \in \mathbb{R}_{\geq 0}^{3 \times 3}$ cujas linhas correspondem aos três modos candidatos (local, RSU, V2V) e cujas

colunas são os três critérios de custo:

$$\mathbf{X} = \begin{bmatrix} \text{JCT}_j^{(0)} & E_j^{(0)} & 0 \\ \text{JCT}_j^{(1)} & E_j^{(1)} & d_{\text{RSU}} \\ \text{JCT}_j^{(2)} & E_j^{(2)} & d_{\text{V2V}} \end{bmatrix}, \quad (9)$$

onde d_{RSU} e d_{V2V} são as distâncias euclidianas ao RSU e ao par V2V selecionado, respectivamente. Se um modo for inviável, sua linha é definida como 10^6 para excluí-lo das soluções ideais.

Normalização ponderada.

Cada coluna j é normalizada por sua norma euclidiana $\|\mathbf{X}_{:,j}\|_2$ e depois multiplicada pelo peso $w_j \in \{0,4; 0,3; 0,3\}$ (JCT, energia, distância), resultando na matriz normalizada ponderada $\tilde{\mathbf{X}}$.

TOPSIS L_1 -Manhattan.

Diferimos do TOPSIS clássico medindo distâncias à solução ideal positiva (PIS) A^+ e à solução ideal negativa (NIS) A^- usando a norma Manhattan (L_1):

$$D_i^+ = \sum_k |\tilde{x}_{ik} - A_k^+|, \quad D_i^- = \sum_k |\tilde{x}_{ik} - A_k^-|. \quad (10)$$

O escore de proximidade é então $\mathcal{C}_i = D_i^- / (D_i^+ + D_i^-)$. Como todos os critérios são objetivos de minimização de custo, A^+ é o mínimo coluna a coluna e A^- é o máximo coluna a coluna de $\tilde{\mathbf{X}}$. A métrica L_1 é menos sensível a valores extremos, propriedade crítica quando as estimativas de fila são ruidosas, e é computacionalmente mais leve que L_2 .

Seleção do rank-1.

O candidato de descarregamento rank-1 é o modo não local viável com o maior $\mathcal{C}_i$:

$$m^* = \underset{i \in \{1,2\}}{\operatorname{argmax}} \mathcal{C}_i \text{ s.t. modo } i \text{ é viável.} \quad (11)$$

Empates são resolvidos em favor do V2V para estimular balanceamento de carga. Se nenhum modo de descarregamento for viável, m^* recorre à execução local independentemente da ação do agente.

Vetor de estado 6D.

O estado $\mathbf{s} \in \mathbb{R}^6$ é montado como:

$$\begin{aligned} s_1 &= C_j / 10^{10} \\ s_2 &= \operatorname{clip}\left(1 - \frac{\text{custo}_{m^*}}{\text{custo}_{\text{loc}}}, -1, 1\right) \\ s_3 &= \min\left(1, \text{JCT}^{(0)} / D_j\right) \\ s_4 &= \min\left(1, \text{JCT}^{(m^*)} / D_j\right) \\ s_5 &= \min\left(1, |Q_{\text{RSU}}| / 25\right) \\ s_6 &= \min\left(1, |Q_{v_0}| / 25\right) \end{aligned} \quad (12)$$

onde os componentes representam: complexidade da tarefa; vantagem do descarregamento; urgência local; urgência rank-1; ocupação da fila RSU; e ocupação da fila local. onde $\text{custo}_m = \alpha \text{JCT}^{(m)} + \beta E^{(m)}$. Todos os componentes pertencem a $[0, 1]$ ou $[-1, 1]$, tornando $\mathbf{s}$ invariante à escala, pois não depende do número de candidatos disponíveis, apenas da qualidade relativa do melhor deles.

4.2 Soft Actor-Critic de Máxima Entropia

O agente SAC otimiza o objetivo de máxima entropia [39]:

$$\pi^* = \underset{\pi}{\operatorname{argmax}} \mathbb{E}_{\tau \sim \pi} \left[\sum_t \gamma^t (r_t + \alpha_T \mathcal{H}[\pi(\cdot | s_t)]) \right], \quad (13)$$

onde α_T é o parâmetro de temperatura ajustado automaticamente. As redes ator e crítico são MLPs com duas camadas ocultas de 256 unidades e ativações ReLU. O ator gera logits para a distribuição de ação de Bernoulli; a temperatura α_T é adaptada para uma entropia alvo de $\log 2/2$ nats. O buffer de replay armazena 10^5 transições; o tamanho do mini-lote é 256; a taxa de aprendizado é 3×10^{-4} para todas as redes.

Por que ação binária?

Delegar a seleção do destino de descarregamento ao TOPSIS reduz o espaço de ações de $|\mathcal{V}(t)| + 2$ dimensões a um único bit. Essa simplificação é sem perda da perspectiva do agente, pois a ordenação relativa dos candidatos já está codificada em $s_2 - s_4$.

4.3 Função de Recompensa Ciente do Prazo

A recompensa r_j para a tarefa τ_j é projetada para guiar o agente ao cumprimento do prazo enquanto fornece um sinal de gradiente contínuo:

$$r_j = \begin{cases} 1,0 - 0,5 \cdot \rho_j & \text{se } \text{JCT}_j \leq D_j, \\ -1,0 - \min(1, \delta_j) & \text{caso contrário,} \end{cases} \quad (14)$$

onde

$$\rho_j = \frac{1}{2} \left(\frac{\text{JCT}_j}{D_j} + \frac{E_j}{E_{\max,j}} \right) \in [0, 1] \quad (15)$$

é uma razão de custo normalizada, e $\delta_j = (\text{JCT}_j - D_j)/D_j$ é a violação relativa do prazo. Execuções bem-sucedidas rendem $r_j \in [0,5, 1,0]$: tarefas mais rápidas e eficientes recebem recompensas maiores. Execuções com falha rendem $r_j \in [-2, -1]$: a penalidade cresce com o grau de atraso, fornecendo um gradiente que ensina o agente a falhar pelo menor tempo possível.

5 Avaliação de Desempenho

5.1 Configuração da Simulação

Todos os algoritmos executam em um simulador de eventos discretos VEC customizado escrito em C++20 com `simcpp20` e a biblioteca de álgebra linear `Armadillo`. As redes neurais são treinadas e exportadas usando `LibTorch` (API C++ do `PyTorch`). Os cenários de mobilidade são gerados com parâmetros V2X realistas, com velocidades veiculares de 30 km h^{-1} a 120 km h^{-1} , largura de banda de uplink 5G de 20 MHz, sidelink V2V de 10 MHz, frequência do RSU $f_{\text{RSU}} = 2 \text{ GHz}$ e frequência embarcada $f_{\text{loc}} \in [0,5; 1,0] \text{ GHz}$. Os prazos das tarefas seguem uma Gaussiana truncada em $[0,05; 2,0] \text{ s}$; complexidade de CPU $C_j \sim \text{Uniforme}[10^8, 10^{10}]$ ciclos.

Treinamento.

Todos os agentes DRL são treinados por 400 episódios em cenários com densidade veicular fixa de 10 tasks/s. Cada episódio de treinamento simula 100 chegadas de tarefas com uma nova realização de mobilidade extraída da mesma distribuição.

Teste.

Após o treinamento, os agentes são avaliados zero-shot em 601 cenários de teste cobrindo cinco regimes de densidade: $\{10, 15, 20, 25, 30\}$ tasks/s. Nenhum aprendizado ou adaptação adicional ocorre durante os testes.

Linhas de base.

Comparamos com: Always-Local (nunca descarrega), Greedy (sempre descarrega ao par V2V com menor JCT estimado, uma linha de base oráculo otimista), Greedy-Fair (greedy com restrição de equidade de Jain), Random (descarrega a um par aleatório), SAC-Base (SAC com vetor de estado bruto de 11 dimensões, sem TOPSIS), PPO-TOPSIS (PPO on-policy com o mesmo estado TOPSIS 6D) e TD3-TOPSIS (TD3 off-policy com o mesmo estado TOPSIS 6D).

Métricas.

- **Taxa de cumprimento de prazo:** fração de tarefas com $\text{JCT}_j \leq D_j$.
- **SWQ:** $\text{SWQ} = \hat{\tau} \cdot \overline{[(D_j - \text{JCT}_j)/D_j]_{\text{sucesso}}}$, que recompensa o cumprimento do prazo com grande folga.
- **JCT médio e energia total consumida.**

5.2 Comparação Geral de Desempenho

A Figura 2 e a Tabela 4 resumem o desempenho agregado em todos os 601 cenários de teste.

O TOPSIS melhora consistentemente todas as famílias DRL.

A Tabela 4 revela um resultado central deste artigo: parear o TOPSIS com qualquer algoritmo DRL testado melhora a taxa de cumprimento de prazo em relação à versão base correspondente. A magnitude do ganho, contudo, varia substancialmente entre as famílias de algoritmos e reflete a interação entre o estado TOPSIS e a estratégia de exploração de cada algoritmo.

SAC experimenta a melhoria mais dramática: o SAC-Base alcança apenas 4,47% de cumprimento porque converge para uma política quase sempre de descarregamento (94,9% V2V), incorrendo em atrasos proibitivos de fila quando os vizinhos estão congestionados. Adicionar o estado TOPSIS resgata a política para 44,36%

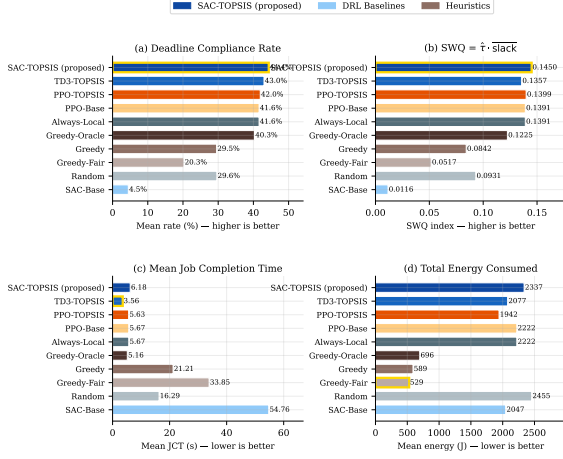


Figura 2 Comparação de desempenho entre as principais métricas. O SAC-TOPSIS (azul escuro) alcança a maior taxa de cumprimento de prazo e o maior índice SWQ. A borda dourada marca o melhor valor em cada gráfico.

(+39,9 pp), o melhor resultado entre todas as políticas avaliadas. A representação 6D fornece o sinal semântico, com razões de urgência e ocupações de fila, necessário para que o SAC precisa para distinguir decisões de descarregamento benéficas das prejudiciais.

TD3 já colapsa para always-local em sua forma base (41,62%, idêntico ao Always-Local), pois sua política determinística evita o caminho arriscado V2V sem pressão de exploração. Com o estado TOPSIS, o TD3 aprende a explorar V2V oportunisticamente (23,4%), elevando o cumprimento para 43,01% (+1,4 pp) e alcançando o menor JCT médio (3,56 s) de todas as políticas.

PPO também colapsa para always-local em sua forma base (41,61%) devido à alta variância do gradiente on-policy. O estado TOPSIS proporciona melhora marginal para 41,95% (+0,3 pp). O ganho é limitado porque a natureza on-policy do PPO o impede de explorar plenamente o estado mais rico, pois a variância nas estimativas de gradiente ainda empurra a política para a ação local segura, apesar do sinal informativo s_2 (vantagem de descarregamento).

No geral, o SAC-TOPSIS alcança a maior taxa de cumprimento de prazo (44,36%) e SWQ (0,1450) entre todas as políticas, incluindo a linha de base oráculo Greedy (40,31%). Os resultados confirmam que o estado TOPSIS é o principal motor de desempenho, e não da escolha do algoritmo

Tabela 4 Desempenho agregado em 601 cenários de teste (média por cenário, densidades $\{10, 15, 20, 25, 30\}$ tasks/s). As setas ($\uparrow$) indicam o ganho ao adicionar TOPSIS a cada algoritmo base. Negrito indica o melhor valor DRL; $\dagger$ indica a linha de base oráculo.

Política	Prazo (%)	SWQ	JCT (s)	Energia (J)
SAC-Base	4,47	0,0116	54,76	2047
SAC-TOPSIS	44,36	0,1450	6,18	2337
$\uparrow +39,9$				
TD3-Base	41,62	0,1391	3,53	2222
TD3-TOPSIS	43,01	0,1357	3,56	2077
$\uparrow +1,4$				
PPO-Base	41,61	0,1391	5,67	2222
PPO-TOPSIS	41,95	0,1399	5,63	1942
$\uparrow +0,3$				
Always-Local	41,61	0,1391	5,67	2222
Greedy-Oracle $\dagger$	40,31	0,1225	5,16	696
Greedy	29,54	0,0842	21,21	589
Greedy-Fair	20,26	0,0517	33,85	529
Random	29,61	0,0931	16,29	2455

DRL, pois as três famílias se beneficiam, e as diferenças residuais entre elas são efeitos secundários de seus respectivos mecanismos de exploração.

5.3 Generalização Zero-Shot

A Figura 3 mostra a taxa de cumprimento de prazo em função da densidade veicular. O SAC-TOPSIS lidera consistentemente em todas as densidades, mantendo $> 34\%$ de cumprimento mesmo no regime de congestionamento inédito mais severo (30 tasks/s), onde o Greedy heurístico cai para apenas 14%. A diferença sobre o Always-Local varia de +1,73 pp em 25 tasks/s a +4,15 pp em 10 tasks/s, confirmando que o benefício de aprender uma política adaptativa é mais pronunciado em menor congestionamento, onde o descarregamento V2V oportunista é mais confiável.

A invariância à escala do vetor de estado 6D é o habilitador fundamental: como s_3 – s_4 são razões

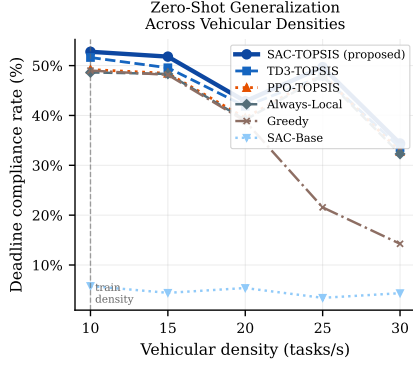


Figura 3 Taxa de cumprimento de prazo em função da densidade veicular. Todos os agentes foram treinados apenas em 10 tasks/s (linha vertical tracejada). O SAC-TOPSIS (círculos sólidos) supera todas as linhas de base consistentemente sem retreinamento específico por densidade.

relativas ao prazo e s_5-s_6 são ocupações de fila normalizadas, a política aprendida pelo agente transfere-se diretamente para ambientes com mais ou menos candidatos. O SAC-Base, em contraste, codifica tamanhos absolutos de fila e valores brutos de frequência que perdem seu significado sob mudança de distribuição.

5.4 Distribuição dos Modos de Execução

A Figura 4 mostra a proporção média de execuções locais, RSU e V2V por política. O SAC-TOPSIS explora o descarregamento V2V (25,9%), que oferece menor latência que o RSU quando um veículo par está próximo. Crucialmente, o módulo TOPSIS garante que o V2V seja escolhido apenas quando oferece uma pontuação TOPSIS melhor que o RSU, impedindo a migração de carga para pares sobrecarregados.

5.5 Convergência do Treinamento

A Figura 5 compara as curvas de treinamento do SAC-TOPSIS, SAC-Base, TD3-TOPSIS e PPO-TOPSIS. Todos os agentes são treinados por 400 episódios no cenário de 10 tasks/s. SAC-TOPSIS e TD3-TOPSIS exibem melhoria de recompensa mais rápida e menor variância que o SAC-Base, confirmando que o estado TOPSIS fornece um sinal de aprendizado mais informativo. O PPO-TOPSIS converge para um ótimo local com menor

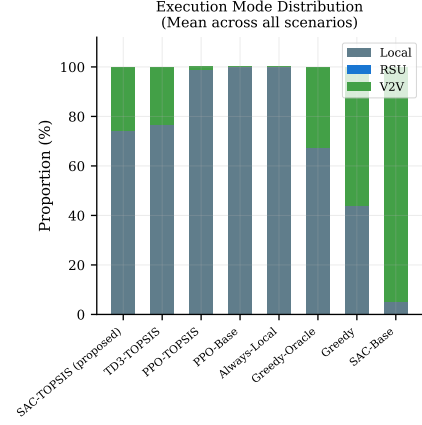


Figura 4 Distribuição dos modos de execução. O SAC-TOPSIS equilibra adaptativamente execução local e V2V, enquanto o SAC-Base sobrecarrega o RSU com descarregamentos.

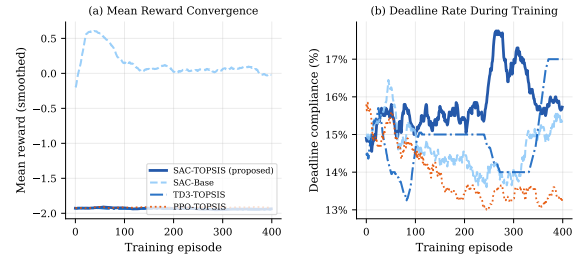


Figura 5 Convergência do treinamento ao longo de 400 episódios. (a) Recompensa média (suavizada com janela de 20 episódios); (b) taxa de cumprimento de prazo durante o treinamento. O SAC-TOPSIS alcança convergência estável em ambas as métricas.

recompensa média, consistente com sua política always-local no tempo de teste.

5.6 Estudo de Ablação: TOPSIS como Primitiva Universal de Engenharia de Estado

A Figura 6 apresenta uma ablação cross-família na qual cada algoritmo DRL é avaliado com seu estado base bruto e com o estado TOPSIS 6D, mantendo todos os demais hiperparâmetros idênticos. As setas que anotam cada par quantificam o ganho atribuível exclusivamente à mudança de representação de estado.

Três achados se destacam. Primeiro, o ganho é monótono, pois o TOPSIS nunca prejudica nenhum algoritmo. Segundo, o ganho é maior para

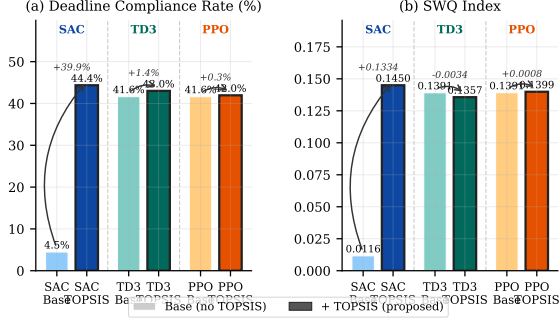


Figura 6 Ablação cross-familia: Base (sem TOPSIS, barras mais claras) vs. +TOPSIS (barras mais escuras com borda preta) para SAC, TD3 e PPO. As setas curvas mostram o ganho ao adicionar TOPSIS a cada família. O ganho é consistente em todas as famílias, confirmando que a compressão de estado TOPSIS é um motor de desempenho universal.

a política base mais instável (SAC-Base colapsa, SAC-TOPSIS recupera), o que confirma que o estado TOPSIS resolve o problema de colapso de representação em vez de apenas ajustar uma política já razoável. Terceiro, o ganho é agnóstico ao algoritmo, pois mesmo o TD3, que tem uma política base bem comportada (ainda que conservadora), melhora +1,4 pp após a mudança de estado. Este último ponto é fundamental, pois mostra que o estado TOPSIS carrega informação útil além do que qualquer algoritmo base pode inferir a partir de observações brutas, e que essa informação é explorável por diferentes paradigmas de aprendizado.

5.7 Ablação Interna do SAC: Decomposição Causal

Para entender quais escolhas de projeto dentro do SAC-TOPSIS mais contribuem para seu desempenho, conduzimos uma ablação refinada na qual cada variante adiciona exatamente um componente em relação ao SAC-Base. A Tabela 5 e a Figura 7 resumem os resultados.

Três perspectivas complementares emergem dessa decomposição.

- (1) A ação binária é a maior alavanca individual (+34,1 pp, SAC-A1). Reduzir o espaço de ações de três classes (LOCAL/RSU/V2V) para uma decisão binária (LOCAL vs. melhor descarregamento rank-1) resolve em grande parte o colapso do SAC-Base. Sem TOPSIS,

Tabela 5 Ablação interna do SAC. Cada linha adiciona uma escolha de projeto sobre o SAC-Base. Δ é o ganho na taxa de prazo em relação ao SAC-Base.

Variante	Mudança	Prazo	Δ
SAC-Base	— (base)	4,47%	—
SAC-A1	Ação binária	38,59%	+34,1
SAC-R1	Recompensa	37,64%	+33,2
SAC-S1	Estado compacto	28,32%	+23,8
SAC-T1	TOPSIS 3-ac	32,10%	+27,6
SAC-TOPSIS	Completo	44,36%	+39,9

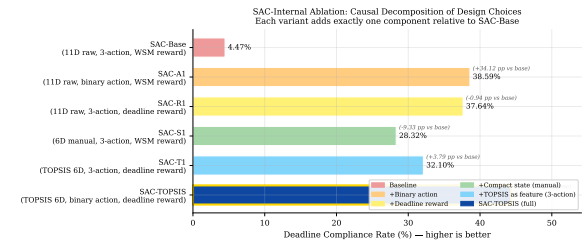


Figura 7 Decomposição causal interna do SAC. Cada barra corresponde a uma variante que isola uma escolha de projeto. Todos os ganhos são reportados em relação ao SAC-Base (4,47%). O SAC-TOPSIS completo combina os três componentes e alcança a maior taxa de cumprimento.

o agente ainda deve avaliar os candidatos com base em características brutas, mas a simplificação da fronteira de decisão já impede a política degenera sempre-V2V.

- (2) A modelagem de recompensa ciente de prazo é igualmente crítica (+33,2 pp, SAC-R1). Substituir a recompensa do Método da Soma Ponderada (WSM) pela recompensa contínua proporcional ao prazo da Equação (14) fornece um sinal de gradiente mesmo no regime de falha, permitindo que o agente aprenda a minimizar o atraso em vez de apenas evitá-lo. Esse ganho é independente da representação de estado, demonstrando que a engenharia de recompensa é uma contribuição autônoma.
- (3) O estado TOPSIS proporciona ganhos compostos sobre qualquer componente isolado. O SAC-S1 mostra que compactar o estado para 6D manualmente (sem TOPSIS) já rende +23,8 pp sobre o SAC-Base, confirmando que a engenharia de features importa. O SAC-T1

mostra que usar o TOPSIS para construir o estado 6D (em vez de features ad-hoc) eleva isso para +27,6 pp, mesmo quando o espaço de ações ainda tem 3 classes. O SAC-TOPSIS completo, que combina o estado TOPSIS com a ação binária e a recompensa de prazo, alcança +39,9 pp, mais que qualquer componente individual, confirmando que as três escolhas de projeto são sinérgicas em vez de meramente aditivas.

6 Conclusão e Trabalhos Futuros

Este artigo apresentou o SAC-TOPSIS, uma arquitetura híbrida para descarregamento de tarefas ciente de prazo em redes VEC. A inovação central é usar o TOPSIS não como tomador de decisão autônomo, mas como um módulo de engenharia de estado que converte um conjunto de candidatos de tamanho variável e multicritério em uma representação fixa 6D invariante à escala. Um agente SAC de máxima entropia treinado sobre essa representação aprende uma política binária local/d Descarregamento que: (i) alcança uma taxa de cumprimento de prazo de 44,36%, a maior entre todas as políticas DRL e heurísticas avaliadas em 601 cenários de teste; (ii) atinge um índice SWQ de 0,1450; (iii) generaliza zero-shot para densidades veiculares inéditas (10–30 tasks/s) sem retreinamento; e (iv) evita os colapsos de modo do PPO-TOPSIS (always-local) e do SAC-Base (always-V2V).

Um estudo de ablação cross-família confirma que a compressão de estado TOPSIS é o principal motor de desempenho, pois melhora o cumprimento de prazo para todas as famílias DRL testadas: SAC (+39,9 pp), TD3 (+1,4 pp) e PPO (+0,3 pp), independentemente de seu algoritmo de aprendizado base. A vantagem residual do SAC sobre o TD3 (+1,35 pp) é um efeito secundário da exploração regularizada por entropia, que impede a política de colapsar para a estratégia conservadora always-local que afeta os métodos determinísticos off-policy em densidades mais altas.

Trabalhos futuros.

Planejamos estender o módulo TOPSIS para Fuzzy-TOPSIS, incorporando incerteza linguística

nos pesos dos critérios para lidar com telemetria atrasada e estimativas ruidosas de canal. Também investigaremos extensões multi-agente nas quais cada veículo em $\mathcal{V}(t)$ executa seu próprio agente SAC-TOPSIS com coordenação descentralizada. Por fim, a avaliação hardware-in-the-loop com rádios sidelink 5G reais está planejada para validar o modelo de energia sob condições físicas de canal.

Referências

- [1] Kandali, K., Nough, S., Bennis, L., Bennis, H.: A comprehensive survey on vanet-iot integration toward the internet of vehicles: Architectures, communications, and system challenges. *Data and Metadata* **4**, 521 (2025) <https://doi.org/10.56294/dm2025521>
- [2] Uddin, A., Zhang, N., Sakr, A.H.: Intelligent offloading in vehicular edge computing: A comprehensive review of deep reinforcement learning approaches and architectures. *arXiv preprint arXiv:2502.06963v2* (2025)
- [3] Zheng, K., Zheng, Q., Chatzimisios, P., Xiang, W., Zhou, Y.: Heterogeneous vehicular networking: A survey on architecture, challenges, and solutions. *IEEE Communications Surveys & Tutorials* **17**(4), 2377–2396 (2015) <https://doi.org/10.1109/COMST.2015.2440103>
- [4] Naik, G., Choudhury, B., Park, J.-M.J.: IEEE 802.11bd & 5g nr v2x: Evolution of radio access technologies for v2x communications. *IEEE Access* **7**, 70113–70128 (2019) <https://doi.org/10.1109/ACCESS.2019.2918458>
- [5] Chen, S., al.: Vehicle-to-everything (v2x) services: Mode 4 communication in lte-v2x. *IEEE Access* **5**, 22461–22471 (2017)
- [6] Federal Communications Commission (FCC): Use of the 5.850-5.925 ghz band: Second report and order. Technical Report FCC 24-110, FCC (November 2024). Adopted November 20, 2024. Establishing sunset for DSRC by December 2025.
- [7] Mao, Y., You, C., Zhang, J., Huang, K., Letaief, K.B.: A survey on mobile edge computing: The communication perspective. *IEEE Communications Surveys & Tutorials* **19**(4), 2322–2358 (2017) <https://doi.org/10.1109/COMST.2017.2745201>

- [8] Miriyala, S., Chirra, V.R.: Deep learning approaches for computation offloading in edge computing: A critical review. *Telecommunication Systems* **89**(42) (2026) <https://doi.org/10.1007/s11235-026-01426-y>
- [9] Liu, Y., Yu, H., Xie, S., Zhang, Y.: Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks. *IEEE Transactions on Vehicular Technology* **68**(11), 11158–11168 (2019) <https://doi.org/10.1109/TVT.2019.2935450>
- [10] Fang, J., Shi, J., Lu, S., Zhang, M.: An efficient computation offloading strategy with mobile edge computing for iot. *Micromachines* **12**(2), 204 (2021) <https://doi.org/10.3390/mi12020204>
- [11] Long, Y., Wang, Z., Lan, S., Zhang, R., Xu, K.: Energy-latency tradeoff for task offloading and resource allocation in vehicular edge computing. *Computer Networks* **258**, 111026 (2025)
- [12] Luo, Q., Luan, T.H., Shi, W., Fan, P.: Heterogeneous task oriented data scheduling in vehicular edge computing via deep reinforcement learning. *IEEE Transactions on Vehicular Technology* **73**(12), 19582–19596 (2024) <https://doi.org/10.1109/TVT.2024.3444815>
- [13] He, Y., Ren, J., Yu, G., Cai, Y.: D2D Communications Meet Mobile Edge Computing for Enhanced Computation Capacity in Cellular Networks. *IEEE Transactions on Wireless Communications* **18**(3), 1750–1763 (2019) <https://doi.org/10.1109/TWC.2019.2896999>
- [14] Wang, M., Yi, H., Jiang, F., Lin, L., Gao, M.: Review on offloading of vehicle edge computing. *Journal of Artificial Intelligence and Technology* **2**(4), 132–143 (2022) <https://doi.org/10.37965/jait.2022.0120>
- [15] Wang, B., Wang, C., Huang, W., Song, Y., Qin, X.: A survey and taxonomy on task offloading for edge-cloud computing. *IEEE Access* **8**, 186080–186101 (2020)
- [16] Chen, S., Li, W., Sun, J., Pace, P., He, L., Fortino, G.: An efficient collaborative task offloading approach based on multi-objective algorithm in mec-assisted vehicular networks. *IEEE Transactions on Vehicular Technology* **74**(7), 11249–11263 (2025) <https://doi.org/10.1109/TVT.2025.3543412>
- [17] Ahmed, M., *et al.*: A survey on vehicular task offloading: Classification, issues and future challenges. *Journal of King Saud University – Computer and Information Sciences* **34**(7), 4135–4162 (2022) <https://doi.org/10.1016/j.jksuci.2022.05.016> . Elsevier / ScienceDirect
- [18] Li, Z., Zhu, Q.: Genetic algorithm-based optimization of offloading and resource allocation in mobile-edge computing. *Information* **11**(2), 83 (2020)
- [19] You, Q., Tang, B.: Efficient task offloading using particle swarm optimization algorithm in edge computing for industrial internet of things. *Journal of Cloud Computing* **10**(1), 41 (2021)
- [20] Kishor, A., Chakarbarty, C.: Task offloading in fog computing for using smart ant colony optimization. *Wireless personal communications* **127**(2), 1683–1704 (2022)
- [21] Wu, Z., Jia, Z., Pang, X., Zhao, S.: Deep reinforcement learning-based task offloading and load balancing for vehicular edge computing. *Electronics* **13**(8), 1511 (2024)
- [22] Cao, C., Guo, Z., Zheng, X.: A survey on task offloading decision-making algorithms in heterogeneous vehicular network. *IEEE Access* **13**, 1–20 (2025)
- [23] Shao, M., *et al.*: Graph neural network-based task offloading and resource allocation for scalable vehicular networks. *IEEE Transactions on Vehicular Technology* (2025). Early Access / Preprint available 2024
- [24] Shi, W., Chen, L., Zhu, X.: Task offloading decision-making algorithm for vehicular edge computing: A deep-reinforcement-learning-based approach. *Sensors* **23**(17), 7595 (2023)
- [25] Li, Y., *et al.*: Deep reinforcement learning for task offloading in vehicular edge computing: A survey. *arXiv preprint arXiv:2501.01234* (2025). Foco em modelagem temporal e dinâmica de VEC
- [26] Sousa Vieira, A.S., Souza, A.B., Celestino Júnior, J.: FOFF: an energy-efficient task offloading in VEC-enabled vehicular networks using fuzzy TOPSIS. *Cluster Computing* (2025) <https://doi.org/10.1007/s10586-025-05027-3>
- [27] Tan, L.T., Hu, R.Q.: Mobility-aware edge caching and computing in vehicle networks: A deep reinforcement learning. *IEEE Transactions on Vehicular Technology* **67**(11), 10190–10203 (2018)

- [28] Pang, H., Wang, Z.: Dueling double deep q network strategy in mec for smart internet of vehicles edge computing networks. *Journal of Grid Computing* **22**, 1–12 (2024)
- [29] Fofana, N., Letaifa, A.B., Rachedi, A.: Intelligent task offloading in vehicular networks: A deep reinforcement learning perspective. *IEEE Transactions on Vehicular Technology* (2024)
- [30] Gao, Z., Yang, L., Dai, Y.: Large-scale computation offloading using a multi-agent reinforcement learning in heterogeneous multi-access edge computing. *IEEE Transactions on Mobile Computing* **22**(6), 3425–3443 (2023)
- [31] Zheng, J., Zhang, Y., Luan, T.H., Mu, P.-K., Li, G., Dong, M., Wu, Y.: Digital twin enabled task offloading for iovs: A learning-based approach. *IEEE Transactions on Network Science and Engineering* **11**, 659–672 (2023)
- [32] Nie, L., Wang, H., Feng, G., Sun, J., Lv, H., Cui, H.: A deep reinforcement learning assisted task offloading and resource allocation approach towards self-driving object detection. *Cloud Computing* **12**, 131 (2023)
- [33] Liu, Z., Huang, L., Gao, Z., Luo, M., Hosseinalipour, S., Dai, H.: Ga-drl: Graph neural network-augmented deep reinforcement learning for dag task scheduling over dynamic vehicular clouds. *IEEE Transactions on Network and Service Management* **21**(4), 4226–4242 (2024)
- [34] Guo, H., et al.: Deep reinforcement learning for task offloading in vec. *IEEE Transactions on Mobile Computing* (2024)
- [35] Silva, V.R.G.d., Valderrama, C., Manneback, P., Xavier-de-Souza, S.: Analytical energy model parametrized by workload, clock frequency and number of active cores for share-memory high-performance computing applications. *Energies* **15**, 1213 (2022) <https://doi.org/10.3390/en15031213>
- [36] Rao, R., Vrudhula, S.: Performance optimal processor throttling under thermal constraints. In: *Proceedings of the 2007 International Conference on Compilers, Architecture, and Synthesis for Embedded Systems*, pp. 257–266 (2007). <https://doi.org/10.1145/1283340.1283383>
- [37] Study on channel model for frequencies from 0.5 to 100 ghz (3gpp tr 38.901 version 16.1.0 release 16). Technical Report ETSI TR 138 901 V16.1.0, 3GPP / ETSI (November 2020). Release 16
- [38] Yan, J.: Towards dependable 5g-nr sidelink communication. Ph.d. dissertation, Sorbonne Université (December 2024). HAL Id: tel-05077398
- [39] Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: *International Conference on Machine Learning*, pp. 1861–1870 (2018). PMLR